Linear classifiers: from the perceptron to the support vector machine

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

A linear classifier separates classes with a hyperplane. Many algorithms learn one — nearest centroid, Gaussian discriminant analysis, least squares, the perceptron, logistic regression, support vector machines — and they differ only in which hyperplane they choose. This note develops the geometry they share, then derives each algorithm step by step, traces it by hand on a few points, gives its pseudocode and its running time (for Gaussian discriminant analysis by reference to Note 6), and ends with what no hyperplane can do. The perceptron receives special attention: its update rule, its convergence proof and its dual form lead directly to the kernel methods of Note 10 and to the neurons of week 5.

1 What makes a classifier linear

A binary classifier is *linear* if its decision function has the form

$$f(\mathbf{x}) = \operatorname{sign}\big(g(\mathbf{x})\big), \qquad g(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b,$$

with labels $t \in \{-1, +1\}$ (the $t = 2y - 1$ of Note 4, Section 2; in this note and in Note 10 the letter t is the class label, while y is kept for model outputs and regression targets), weight vector $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$; g is the *linear discriminant function* or *score*. For m classes, $f(\mathbf{x}) = \arg\max_c g_c(\mathbf{x})$ with $g_c(\mathbf{x}) = \mathbf{w}_c^{\mathsf{T}}\mathbf{x} + b_c$, that is $f(\mathbf{x}) = \arg\max_c (W\mathbf{x} + \mathbf{b})_c$ with the $\mathbf{w}_c^{\mathsf{T}}$ as rows of $W \in \mathbb{R}^{m\times d}$. For $m = 2$ the two forms agree: class 2 wins exactly when $\mathbf{w}_2^{\mathsf{T}}\mathbf{x}+b_2 > \mathbf{w}_1^{\mathsf{T}}\mathbf{x}+b_1$, i.e. when $(\mathbf{w}_2-\mathbf{w}_1)^{\mathsf{T}}\mathbf{x}+(b_2-b_1) > 0$, a binary rule with $\mathbf{w} = \mathbf{w}_2 - \mathbf{w}_1$ and $b = b_2 - b_1$. Only differences of scores matter.

Geometry. The decision boundary $H = \{\mathbf{x} : \mathbf{w}^{\mathsf{T}}\mathbf{x} + b = 0\}$ is a hyperplane — a line in two dimensions, a plane in three. It splits $\mathbb{R}^d$ into the two half-spaces $\mathscr{R}_+ = \{\mathbf{x} : g(\mathbf{x}) > 0\}$ and $\mathscr{R}_- = \{\mathbf{x} : g(\mathbf{x}) < 0\}$, the decision regions of the two classes. Two facts are used throughout the note.

1. $\mathbf{w}$ *is normal to* H. If $\mathbf{x}_A$ and $\mathbf{x}_B$ both lie on H, subtracting $\mathbf{w}^{\mathsf{T}}\mathbf{x}_A + b = 0$ from $\mathbf{w}^{\mathsf{T}}\mathbf{x}_B + b = 0$ gives $\mathbf{w}^{\mathsf{T}}(\mathbf{x}_B - \mathbf{x}_A) = 0$: $\mathbf{w}$ is orthogonal to every direction lying inside H.
2. g *measures distance.* Write any point as its orthogonal projection $\mathbf{x}_\perp$ onto H plus a step r along the unit normal, $\mathbf{x} = \mathbf{x}_\perp + r\,\mathbf{w}/\|\mathbf{w}\|$. Applying g and using $g(\mathbf{x}_\perp) = 0$, $g(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x}_\perp + b + r\,\mathbf{w}^{\mathsf{T}}\mathbf{w}/\|\mathbf{w}\| = r\,\|\mathbf{w}\|$. Hence the signed distance of $\mathbf{x}$ to H is

$$r(\mathbf{x}) = \frac{\mathbf{w}^{\mathsf{T}}\mathbf{x} + b}{\|\mathbf{w}\|}, \tag{1}$$

   positive on the side $\mathbf{w}$ points to. The origin is at signed distance $b/\|\mathbf{w}\|$.

Multiplying $(\mathbf{w}, b)$ by any $c > 0$ changes neither H nor the classifier, only the scale of g; this freedom is used to define the SVM in Section 5. In the multiclass case each region $\{\mathbf{x} : g_c(\mathbf{x}) > g_{c'}(\mathbf{x})\ \forall c' \neq c\}$

is an intersection of half-spaces, hence convex: each condition $g_c(\mathbf{x}) > g_{c'}(\mathbf{x})$ reads $(\mathbf{w}_c - \mathbf{w}_{c'})^\mathsf{T}\mathbf{x} + (b_c - b_{c'}) > 0$, which is itself a half-space.

> **Worked example — distance to a line**
>
> For $\mathbf{w} = (1, 2)$, $b = -4$ (the line $x_1 + 2x_2 = 4$) and $\mathbf{x} = (3, 3)$: $g(\mathbf{x}) = 5 > 0$, class $+1$, at distance $5/\sqrt{5} \approx 2.24$; the origin is at $-4/\sqrt{5} \approx -1.79$. Doubling $(\mathbf{w}, b)$ doubles $g(\mathbf{x})$ but not the distances.

The bias trick. As in Note 8, Section 1, we often absorb the bias: with $\tilde{\mathbf{x}} = (1, x_1, \dots, x_d)^\mathsf{T}$ and $\tilde{\mathbf{w}} = (b, w_1, \dots, w_d)^\mathsf{T}$ we have $g(\mathbf{x}) = \tilde{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}}$. A hyperplane through the origin in $\mathbb{R}^{d+1}$ thus represents any hyperplane in $\mathbb{R}^d$, and algorithms can be written without a separate bias.

Linear separability. A dataset is *linearly separable* if some hyperplane classifies all of it correctly: there exist $\mathbf{w}, b$ with $t_i(\mathbf{w}^\mathsf{T}\mathbf{x}_i + b) > 0$ for every i. The quantity $t_i\, g(\mathbf{x}_i)$, the *margin* of example i, is positive exactly when the example is correctly classified, and a large negative value means a confidently wrong prediction; it appears in every loss of this note. Many datasets are not separable; the simplest one, XOR, is discussed in Section 6.

Label conventions. Binary labels are $t \in \{-1, +1\}$ everywhere except in Section 4, where the probabilistic model is more natural with $t \in \{0, 1\}$. With m classes the label is $t \in \{1, \dots, m\}$. The two class means are written $\boldsymbol{\mu}_+$ and $\boldsymbol{\mu}_-$ (the $\boldsymbol{\mu}_1$ and $\boldsymbol{\mu}_0$ of Note 6).

2 Many roads to a hyperplane

The algorithms below share the hypothesis class — every one of them returns a linear decision function — but not the loss, so on the same data each settles on its own line, and these lines perform differently on new data. Figure 1 shows five of them on one dataset.

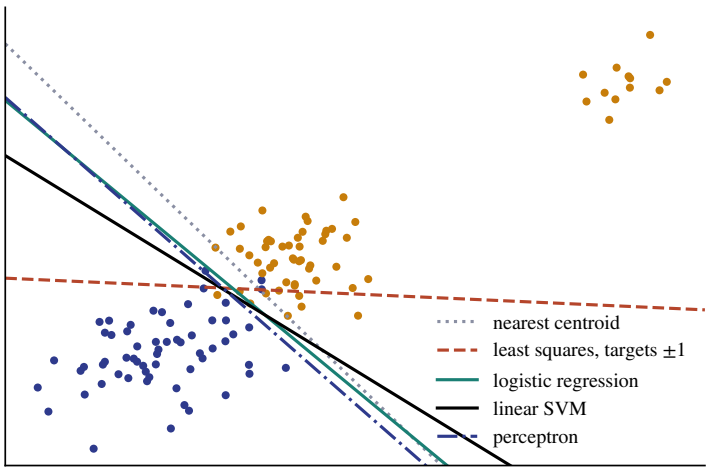


Figure 1. Five linear classifiers on the same 122 points. All boundaries are straight lines, but they differ. Training errors: nearest centroid 4.9%, least squares 9.8%, logistic regression, linear SVM and perceptron 3.3% (the perceptron's last iterate after 100 epochs; the data are not separable). The least-squares line is pulled by the distant amber cluster in the top right.

2.1 Nearest centroid

Store the class means $\boldsymbol{\mu}_c = \frac{1}{n_c}\sum_{i:t_i=c}\mathbf{x}_i$, where n_c is the number of training examples of class c, and assign $\mathbf{x}$ to the nearest. Expanding the squared distance,

$$\|\mathbf{x} - \boldsymbol{\mu}_c\|^2 = \|\mathbf{x}\|^2 - 2\boldsymbol{\mu}_c^\mathsf{T}\mathbf{x} + \|\boldsymbol{\mu}_c\|^2,$$

and $\|\mathbf{x}\|^2$ is the same for all classes, so minimising the distance is maximising $g_c(\mathbf{x}) = 2\boldsymbol{\mu}_c^{\mathsf{T}}\mathbf{x} - \|\boldsymbol{\mu}_c\|^2$: the rule is linear with $\mathbf{w}_c = 2\boldsymbol{\mu}_c$ and $b_c = -\|\boldsymbol{\mu}_c\|^2$. For two classes, halving the difference of the two scores,

$$g(\mathbf{x}) = \tfrac{1}{2}\big(\,\|\mathbf{x}-\boldsymbol{\mu}_-\|^2 - \|\mathbf{x}-\boldsymbol{\mu}_+\|^2\,\big) = (\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-)^{\mathsf{T}}\mathbf{x} - \tfrac{1}{2}\big(\,\|\boldsymbol{\mu}_+\|^2 - \|\boldsymbol{\mu}_-\|^2\,\big);$$

the boundary is the perpendicular bisector of the segment joining the means. The method ignores the shape and spread of the classes, which is why it misplaces the line in Figure 1.

> **Worked example — nearest centroid by hand**
>
> Class A has points $(1,2)$, $(2,3)$, $(3,1)$ and class B has $(5,5)$, $(6,4)$, $(7,6)$. The centroids are $\boldsymbol{\mu}_A = (2,2)$ and $\boldsymbol{\mu}_B = (6,5)$, with $\|\boldsymbol{\mu}_A\|^2 = 8$ and $\|\boldsymbol{\mu}_B\|^2 = 61$. With B as the positive class, $\mathbf{w} = \boldsymbol{\mu}_B - \boldsymbol{\mu}_A = (4,3)$ and $b = -\tfrac{1}{2}(61-8) = -26.5$, so $g(\mathbf{x}) = 4x_1 + 3x_2 - 26.5$. For the query $\mathbf{x} = (4,3)$, $g = 16 + 9 - 26.5 = -1.5 < 0$: class A. Check: $\|\mathbf{x}-\boldsymbol{\mu}_A\|^2 = 5$ and $\|\mathbf{x}-\boldsymbol{\mu}_B\|^2 = 8$, and indeed $\tfrac{1}{2}(5-8) = -1.5$. All six training points are classified correctly.

2.2 Gaussian discriminant analysis and Fisher's direction

With Gaussian classes sharing one covariance Σ, the Bayes classifier is linear, with $\mathbf{w} = \Sigma^{-1}(\boldsymbol{\mu}_+ - \boldsymbol{\mu}_-)$: this is *linear discriminant analysis* (LDA, Note 6, Section 3, which also works out a two-dimensional example by hand). With equal priors it is nearest centroid in the Mahalanobis metric, and it reduces to plain nearest centroid when moreover $\Sigma \propto I$; unequal priors $P_+ \neq P_-$ shift the threshold b by $\log(P_+/P_-)$ but leave $\mathbf{w}$ unchanged. If each class has its own covariance, the boundary becomes quadratic (QDA, Note 6, Section 3) and the classifier is no longer linear, so it falls outside this note.

Fisher [8] reached the same direction without any probability model. Project the data on $\mathbf{w}$ and compare the squared distance between the projected class means, $(\mathbf{w}^{\mathsf{T}}\Delta)^2$ with $\Delta = \boldsymbol{\mu}_+ - \boldsymbol{\mu}_-$, with the projected spread within the classes, $\mathbf{w}^{\mathsf{T}}S_W\mathbf{w}$, where

$$S_W = \sum_{c\in\{+,-\}} \sum_{i\,:\,t_i=c} (\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^{\mathsf{T}}$$

is the *pooled within-class scatter matrix*; S_W/n is the pooled covariance estimate $\hat{\Sigma}$ of Note 6 (dividing by $n-2$ instead gives the unbiased version). Fisher maximises the ratio $F(\mathbf{w}) = (\mathbf{w}^{\mathsf{T}}\Delta)^2/(\mathbf{w}^{\mathsf{T}}S_W\mathbf{w})$. Using $\nabla(\mathbf{w}^{\mathsf{T}}\Delta)^2 = 2(\mathbf{w}^{\mathsf{T}}\Delta)\,\Delta$ and $\nabla(\mathbf{w}^{\mathsf{T}}S_W\mathbf{w}) = 2S_W\mathbf{w}$ (as S_W is symmetric), the quotient rule gives

$$\nabla F = \frac{2(\mathbf{w}^{\mathsf{T}}\Delta)\,\Delta\,(\mathbf{w}^{\mathsf{T}}S_W\mathbf{w}) - (\mathbf{w}^{\mathsf{T}}\Delta)^2\,2S_W\mathbf{w}}{(\mathbf{w}^{\mathsf{T}}S_W\mathbf{w})^2},$$

and $\nabla F = 0$ requires the numerator to vanish, i.e. $S_W\mathbf{w} = c\,\Delta$ with the scalar $c = (\mathbf{w}^{\mathsf{T}}S_W\mathbf{w})/(\mathbf{w}^{\mathsf{T}}\Delta)$. Hence $\mathbf{w} \propto S_W^{-1}\Delta \propto \hat{\Sigma}^{-1}\Delta$, the LDA direction (the length of $\mathbf{w}$ does not matter, since F is unchanged by rescaling $\mathbf{w}$); the threshold b is chosen afterwards, for example midway between the projected means. Computing it is the LDA fit of Note 6, Section 3 ("Fitting and cost"), where the pseudocode and the running time are given: $O(nd^2)$ to form S_W and $O(d^3)$ to solve $S_W\mathbf{w} = \Delta$.

2.3 Least squares on ± 1 targets

The labels $t_i = \pm 1$ are numbers, so linear regression (Note 8, Section 2) can be fitted to them directly, and the sign of the fitted value used as the predicted class: $\tilde{\mathbf{w}} = (\tilde{X}^{\mathsf{T}}\tilde{X})^{-1}\tilde{X}^{\mathsf{T}}\mathbf{t}$, where $\tilde{X}$ is the $n\times(d+1)$ matrix with rows $\tilde{\mathbf{x}}_i^{\mathsf{T}}$ (Note 8, Section 1) and the targets $t_i = \pm 1$ — called y_i in Note 8 — are stacked in $\mathbf{t}$. Ridge regression (Note 8, Section 4) can be used as well; it changes the closed form to $(\tilde{X}^{\mathsf{T}}\tilde{X}+\lambda I)^{-1}\tilde{X}^{\mathsf{T}}\mathbf{t}$ but not the defect discussed next. For m classes, regress the one-hot vectors and take the largest

output. The method is fast and has a closed form, but its loss is wrong for classification. Written in terms of the margin $z_i = t_i\, g(\mathbf{x}_i)$, and using $g(\mathbf{x}_i) - t_i = t_i\big(t_i\, g(\mathbf{x}_i) - 1\big)$, which holds because $t_i^2 = 1$,

$$(g(\mathbf{x}_i) - t_i)^2 = t_i^2\,(t_i\, g(\mathbf{x}_i) - 1)^2 = (1 - z_i)^2,$$

which grows again for $z_i > 1$: a point that is correctly classified *with a large margin* is penalised as if it were an error (Figure 2). A distant cluster of easy examples therefore pulls the boundary towards itself. In Figure 1 the twelve far-away amber points tilt the least-squares line and triple its training error (9.8%) compared with logistic regression (3.3%); fitted without those twelve points, least squares also makes 3.3% errors on all 122.

Worked example — one easy point moves the boundary

In one dimension, take $x = -2, -1$ with $t = -1$ and $x = 1, 2$ with $t = +1$. By symmetry the fit $y = wx + b$ has $b = 0$, $w = \sum x_i t_i / \sum x_i^2 = 6/10$, and the boundary is at $x = 0$, midway between the classes. Now add one positive point at $x = 10$, as correct as a point can be. The normal equations (Note 8) with $n = 5$, $\sum x_i = 10$, $\sum x_i^2 = 110$, $\sum t_i = 1$, $\sum x_i t_i = 16$ read

$$\begin{pmatrix} 5 & 10 \\ 10 & 110 \end{pmatrix}\begin{pmatrix} b \\ w \end{pmatrix} = \begin{pmatrix} 1 \\ 16 \end{pmatrix} \quad\Rightarrow\quad b = -\tfrac{1}{9}, \quad w = \tfrac{7}{45}, \quad \text{boundary } x = -b/w = \tfrac{5}{7} \approx 0.71.$$

The boundary moved by 0.71 towards the positive class, because the fit tries to bring the prediction at $x = 10$ (here $13/9 \approx 1.44$) down to 1 by flattening the line. With ten copies of the point at $x = 10$ the boundary reaches $x \approx 1.03$ and the positive point at $x = 1$ becomes an error. Logistic regression and the SVM, whose losses vanish (or almost) for large margins, are hardly affected.

Algorithm 1 Nearest centroid and the least-squares classifier.

function TrainCentroid($\{(\mathbf{x}_i, t_i)\}_{i=1}^n$, classes $1, \dots, m$)
  for $c = 1, \dots, m$ **do** $\boldsymbol{\mu}_c \leftarrow$ mean of the $\mathbf{x}_i$ with $t_i = c$; $\mathbf{w}_c \leftarrow 2\boldsymbol{\mu}_c$; $b_c \leftarrow -\|\boldsymbol{\mu}_c\|^2$
  end for
  return $\{(\mathbf{w}_c, b_c)\}$ ▷ predict $\arg\max_c(\mathbf{w}_c^{\mathsf{T}}\mathbf{x} + b_c)$
end function
function TrainLeastSquares($\tilde{X} \in \mathbb{R}^{n\times(d+1)}$, targets T (± 1 vector, or one-hot $n \times m$))
  return $\tilde{W} \leftarrow$ LstSq($\tilde{X}$, T) ▷ QR or SVD (Note 8); $\tilde{W}$: $(d+1) \times m$, or $\tilde{\mathbf{w}}$
  predict $\operatorname{sign}(\tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}})$, or $\arg\max_c(\tilde{W}^{\mathsf{T}}\tilde{\mathbf{x}})_c$
end function

Running time — nearest centroid and least-squares classifier

Nearest centroid: training $O(nd)$, one pass to sum the points of each class; prediction $O(md)$ per query, one inner product per class; memory $O(md)$ for the centroids. It is the cheapest classifier in this note. *Least squares*: training $O(nd^2)$ for the QR factorisation of the $n\times(d+1)$ matrix, plus $O(ndm)$ for m one-hot targets; through the normal equations it is $O(nd^2)$ to form $\tilde{X}^{\mathsf{T}}\tilde{X}$ and $O(d^3)$ to solve, which only matters for d in the thousands. Prediction $O(md)$; memory $O(nd)$ during training, $O(md)$ for the model.

2.4 Why not minimise the error rate directly

The training error rate $\frac{1}{n}\sum_i \mathbf{1}\{t_i\, g(\mathbf{x}_i) \le 0\}$ is piecewise constant in $(\mathbf{w}, b)$: a small change of the parameters changes no prediction, so its gradient is zero almost everywhere and gradient descent (Note 7) cannot move; minimising it exactly is a hard combinatorial problem. The remaining algorithms minimise the *surrogate* losses of the margin $z = t\, g(\mathbf{x})$ introduced in Note 4, Section 2 — the squared

loss $(1-z)^2$ (least squares, above), the hinge $\max(0, 1-z)$ (the SVM, Section 5) and the logistic loss $\log(1+e^{-z})$ (logistic regression, Section 4) — plus one more that is new here: the *perceptron loss* $\max(0, -z)$ (Section 3), zero for every correct point and growing linearly with the depth of a mistake. Figure 2 adds it to the curves of Note 4: unlike the hinge it asks for no safety margin, and unlike the squared loss it never penalises a correct point.

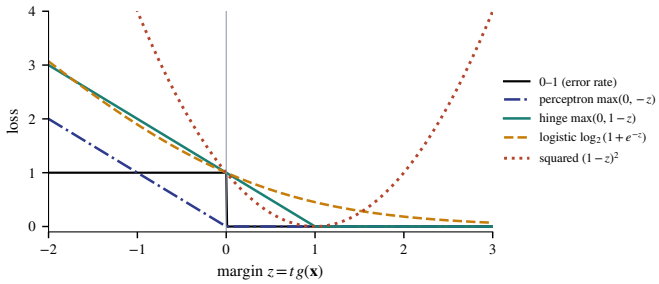


Figure 2. The surrogate losses of Note 4, Section 2, as functions of the margin $z = t\,g(\mathbf{x})$, with the perceptron loss $\max(0, -z)$ added; $z > 0$ means a correct classification. The logistic loss is in base 2.

3 The perceptron

Rosenblatt's perceptron [15], first simulated on a computer in 1957–58 and then built in hardware as the Mark I Perceptron (publicly demonstrated in 1960), was one of the first artificial neurons trained from data.

3.1 The algorithm and its logic

The rule is: look at one example at a time; if it is classified correctly, do nothing; if not, add it to the weights (or subtract it, for a negative example). Algorithm 2 states it with the bias written separately. A score of exactly zero counts as a mistake, which is what gets the algorithm started from $\mathbf{w} = \mathbf{0}$.

Algorithm 2 The perceptron (primal form), with the averaged and pocket variants of Section 3.5.

Require: examples $(\mathbf{x}_i, t_i)$, $t_i \in \{-1, +1\}$; maximum number of epochs E
Ensure: weights $\mathbf{w}$ and bias b (or their averages $\bar{\mathbf{w}}$, $\bar{b}$, or the pocket weights)

$\mathbf{w} \leftarrow \mathbf{0}$, $b \leftarrow 0$; $\mathbf{s} \leftarrow \mathbf{0}$, $s_b \leftarrow 0$, $N \leftarrow 0$ ▷ sums for the averaged variant
pocket $\leftarrow (\mathbf{w}, b)$, *best* $\leftarrow n$ ▷ pocket variant only
for epoch $= 1, \dots, E$ **do**
  mistakes $\leftarrow 0$
  for each i, in random order **do** ▷ cyclic order in the trace below
    if $t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) \le 0$ **then** ▷ wrong side, or on the boundary
      $\mathbf{w} \leftarrow \mathbf{w} + t_i\mathbf{x}_i$, $b \leftarrow b + t_i$, *mistakes* $\leftarrow$ *mistakes* $+ 1$
    end if
    $\mathbf{s} \leftarrow \mathbf{s} + \mathbf{w}$, $s_b \leftarrow s_b + b$, $N \leftarrow N + 1$ ▷ averaged variant only
  end for
  $e \leftarrow$ number of i with $t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) \le 0$ ▷ pocket only; one extra pass, $O(nd)$
  if $e <$ *best* **then** *pocket* $\leftarrow (\mathbf{w}, b)$, *best* $\leftarrow e$
  end if
  if *mistakes* $= 0$ **then break** ▷ all training points correct
  end if
end for
return $(\mathbf{w}, b)$; or $(\bar{\mathbf{w}}, \bar{b}) = (\mathbf{s}/N,\ s_b/N)$ (averaged); or *pocket*

What one update does. In the augmented notation the update is $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} + t_i \tilde{\mathbf{x}}_i$. The new margin of the example that triggered it is

$$t_i\,(\tilde{\mathbf{w}} + t_i\tilde{\mathbf{x}}_i)^{\mathsf{T}}\tilde{\mathbf{x}}_i = t_i\,\tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i + t_i^2\,\|\tilde{\mathbf{x}}_i\|^2 = t_i\,\tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i + \|\mathbf{x}_i\|^2 + 1, \tag{2}$$

so every update raises the margin of the offending example by exactly $\|\mathbf{x}_i\|^2 + 1 > 0$ (the $+1$ comes from the constant feature, i.e. from the bias). It moves the example towards the correct side, though not necessarily all the way across: one update is a correction, not a guarantee. Geometrically, for a positive example $\mathbf{w}$ is rotated *towards* $\mathbf{x}_i$ and for a negative one away from it; the boundary, perpendicular to $\mathbf{w}$, turns accordingly (Figure 3a). The price is that other examples' margins change too, by $t_j t_i\,\tilde{\mathbf{x}}_j^{\mathsf{T}}\tilde{\mathbf{x}}_i$, which may be negative: an update can spoil a point that was correct.

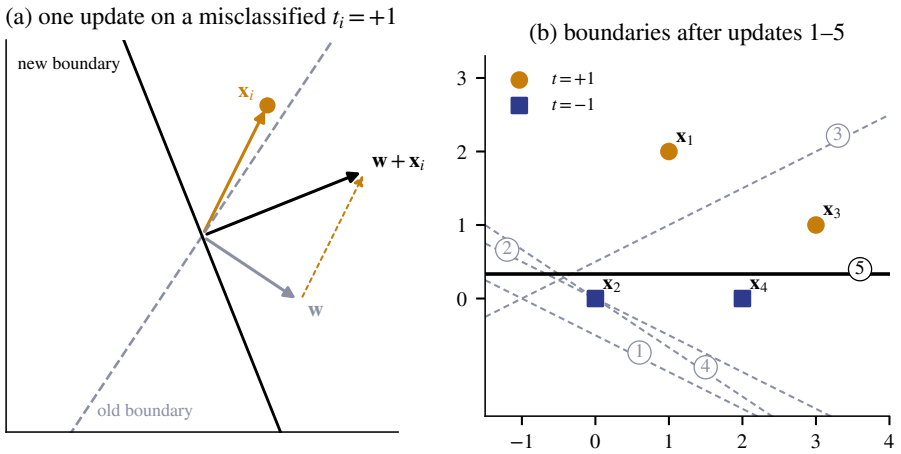


Figure 3. (a) One perceptron update without bias: $\mathbf{x}_i$ (amber) is positive but $\mathbf{w}^{\mathsf{T}}\mathbf{x}_i = -0.5 < 0$. Adding $\mathbf{x}_i$ to $\mathbf{w}$ rotates the weight vector towards it, and the new score is $-0.5 + \|\mathbf{x}_i\|^2 = 4.5$. (b) The boundaries after each of the five updates of the trace in Table 1 (updates 1–5 happen at steps 1, 2, 4, 7, 8); the fifth (solid) separates the four points.

3.2 The perceptron criterion

The update is stochastic gradient descent (Note 7, Section 5) on the *perceptron criterion*

$$J_{\mathrm{p}}(\mathbf{w}, b) = \sum_{i=1}^{n} \max\left(0,\, -t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b)\right) = \sum_{i \in \mathscr{M}} -t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b),$$

where $\mathscr{M}$ is the set of misclassified examples; each term is proportional to the distance of a misclassified point to the boundary (1). For $i \in \mathscr{M}$ the term is linear in the parameters, with derivatives

$$\frac{\partial}{\partial \mathbf{w}}\big[-t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b)\big] = -t_i\mathbf{x}_i, \qquad \frac{\partial}{\partial b}\big[-t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b)\big] = -t_i,$$

and zero for a correct example. A stochastic gradient step on the term of one example is therefore $\mathbf{w} \leftarrow \mathbf{w} + \eta\, t_i\mathbf{x}_i$, $b \leftarrow b + \eta\, t_i$ if the example is wrong, and nothing otherwise. The step size does not matter: starting from zero, every iterate with step η is η times the iterate with step 1, and the signs of all scores — hence all decisions — are the same. This is why the classical algorithm simply uses $\eta = 1$. Descending the full gradient of J_{p} instead gives the *batch perceptron*, $\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_{i \in \mathscr{M}} t_i\mathbf{x}_i$, $b \leftarrow b + \eta \sum_{i \in \mathscr{M}} t_i$, one step per pass over the data at $O(nd)$; restricting the sum to the misclassified points of a mini-batch gives the mini-batch version. Note that J_{p} is also zero at the useless point $\mathbf{w} = \mathbf{0}$, $b = 0$; treating a zero score as a mistake is what keeps the algorithm away from it.

3.3 A complete trace

Take the four points of Figure 3b: $\mathbf{x}_1 = (1,2)$ and $\mathbf{x}_3 = (3,1)$ with $t = +1$, $\mathbf{x}_2 = (0,0)$ and $\mathbf{x}_4 = (2,0)$ with $t = -1$. Their augmented squared norms $\|\mathbf{x}_i\|^2 + 1$ are $6, 1, 11, 5$. Visiting the points in the order $1, 2, 3, 4, 1, 2, \ldots$ from $\mathbf{w} = (0,0)$, $b = 0$ gives Table 1.

Table 1. Perceptron trace. The score $g(\mathbf{x}_i) = \mathbf{w}^\mathsf{T}\mathbf{x}_i + b$ is computed with the parameters *before* the step; $(\mathbf{w}, b)$ are the values *after* it.

step	epoch	example	t_i	$g(\mathbf{x}_i)$	mistake?	update	new $\mathbf{w}$	new b
1	1	$\mathbf{x}_1 = (1,2)$	$+1$	0	yes	1	$(1,2)$	1
2	1	$\mathbf{x}_2 = (0,0)$	-1	1	yes	2	$(1,2)$	0
3	1	$\mathbf{x}_3 = (3,1)$	$+1$	5	no	—	$(1,2)$	0
4	1	$\mathbf{x}_4 = (2,0)$	-1	2	yes	3	$(-1,2)$	-1
5	2	$\mathbf{x}_1$	$+1$	2	no	—	$(-1,2)$	-1
6	2	$\mathbf{x}_2$	-1	-1	no	—	$(-1,2)$	-1
7	2	$\mathbf{x}_3$	$+1$	-2	yes	4	$(2,3)$	0
8	2	$\mathbf{x}_4$	-1	4	yes	5	$(0,3)$	-1
9–12	3	$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \mathbf{x}_4$		$5, -1, 2, -1$	no	—	$(0,3)$	-1

Reading the trace. Each mistake raises the margin of its own example by $\|\mathbf{x}_i\|^2 + 1$, as in (2): at step 1 from 0 to 6, at step 4 from -2 to 3, at step 7 from -2 to 9, at step 8 from -4 to 1. At step 2 the margin of $\mathbf{x}_2$ rises from -1 to 0 only, since $\|\tilde{\mathbf{x}}_2\|^2 = 1$: the point is still not correct, and is fixed only later, through the updates on other points. Step 4 corrects $\mathbf{x}_4$ but spoils $\mathbf{x}_3$ (its score falls from 5 to -2), which costs step 7, and step 7 in turn spoils $\mathbf{x}_4$. After five mistakes the third epoch is clean and the algorithm stops with $g(\mathbf{x}) = 3x_2 - 1$: the horizontal line $x_2 = 1/3$. It separates the data, but passes at distance $1/3$ from $\mathbf{x}_2$ and $\mathbf{x}_4$ while the positive points are far away — the perceptron is satisfied with *any* separating line. Another visiting order gives another line.

3.4 Convergence on separable data

Theorem (Novikoff [13]). Suppose $\|\tilde{\mathbf{x}}_i\| \le R$ for all i and the data are separable with margin $\gamma > 0$ in the augmented space: some unit vector $\tilde{\mathbf{w}}^*$ has $t_i\, \tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{x}}_i \ge \gamma$ for all i. Then the perceptron, started at zero and visiting the examples in any order, makes at most $(R/\gamma)^2$ mistakes.

Proof sketch. Let $\tilde{\mathbf{w}}_k$ be the weights after k mistakes, $\tilde{\mathbf{w}}_0 = \mathbf{0}$, and suppose mistake $k+1$ occurs on example i. Two inequalities follow from the update $\tilde{\mathbf{w}}_{k+1} = \tilde{\mathbf{w}}_k + t_i\tilde{\mathbf{x}}_i$.

1. The alignment with $\tilde{\mathbf{w}}^*$ grows linearly: $\tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{w}}_{k+1} = \tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{w}}_k + t_i\, \tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{x}}_i \ge \tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{w}}_k + \gamma$, hence $\tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{w}}_k \ge k\gamma$.
2. The length grows at most like $\sqrt{k}$: $\|\tilde{\mathbf{w}}_{k+1}\|^2 = \|\tilde{\mathbf{w}}_k\|^2 + 2t_i\, \tilde{\mathbf{w}}_k^\mathsf{T}\tilde{\mathbf{x}}_i + \|\tilde{\mathbf{x}}_i\|^2 \le \|\tilde{\mathbf{w}}_k\|^2 + R^2$, because $t_i\, \tilde{\mathbf{w}}_k^\mathsf{T}\tilde{\mathbf{x}}_i \le 0$ on a mistake. Hence $\|\tilde{\mathbf{w}}_k\|^2 \le kR^2$.

By the Cauchy–Schwarz inequality, $k\gamma \le \tilde{\mathbf{w}}^{*\mathsf{T}}\tilde{\mathbf{w}}_k \le \|\tilde{\mathbf{w}}^*\|\,\|\tilde{\mathbf{w}}_k\| \le \sqrt{k}\,R$, so $\sqrt{k} \le R/\gamma$ and $k \le (R/\gamma)^2$. The intuition: the weights keep turning towards $\tilde{\mathbf{w}}^*$ by a fixed amount per mistake, while their length grows more slowly; the cosine between $\tilde{\mathbf{w}}_k$ and $\tilde{\mathbf{w}}^*$ would exceed 1 after more than $(R/\gamma)^2$ mistakes. □

The bound depends on neither n nor d, only on the geometry of the data. In the trace above, $R^2 = \max_i \|\tilde{\mathbf{x}}_i\|^2 = 11$, and $\tilde{\mathbf{w}}^* = (-1, 0, 2)/\sqrt{5}$ (that is $b = -1$, $\mathbf{w} = (0,2)$, rescaled) gives every point a

margin of at least $\gamma = 1/\sqrt{5} \approx 0.45$; the bound is $11 \times 5 = 55$ mistakes, and five were needed. Note that γ is measured in the augmented space, where $(b, \mathbf{w})$ is normalised as one vector, so it is not the Euclidean distance seen in the plot: the line $x_2 = 1/2$ of this $\tilde{\mathbf{w}}^*$ is at distance $1/2$ from the closest points. The bound is loose, but it says something important: well-separated data (large γ) are learned with few mistakes.

3.5 Non-separable data: pocket and averaged perceptron

If the data are not separable, some example is always misclassified and the perceptron never stops; its weights keep jumping, and the final iterate can be poor even when some earlier one was good. On the data of Figure 1 it made 606 corrections in 100 passes. Two classical remedies return a better, more stable answer. Gallant's *pocket algorithm* [10] keeps "in its pocket" the weights that survived the longest run of consecutive correct classifications; in its *ratchet* version the pocket is replaced only if the training error is also lower. Algorithm 2 uses the simplest form of the idea: once per epoch it counts the training errors and keeps the best weights seen so far, at the cost of one extra pass over the data. The *averaged perceptron* returns the average of the weight vectors over all steps (the extra lines of Algorithm 2): weights that survived many steps without a mistake count more, and the average fluctuates far less than the last iterate. It is the practical approximation of the *voted perceptron* of Freund and Schapire [9], which in their experiments came close to an SVM at the cost of a perceptron.

3.6 The dual form

Since the weights start at zero and only ever add $t_i \tilde{\mathbf{x}}_i$, after training

$$\tilde{\mathbf{w}} = \sum_{i=1}^{n} \alpha_i t_i \, \tilde{\mathbf{x}}_i, \qquad \alpha_i = \text{number of mistakes made on example } i. \tag{3}$$

In the trace, $\alpha = (1, 1, 1, 2)$ and indeed $1 \cdot (1, 1, 2) - 1 \cdot (1, 0, 0) + 1 \cdot (1, 3, 1) - 2 \cdot (1, 2, 0) = (-1, 0, 3)$, i.e. $b = -1$, $\mathbf{w} = (0, 3)$. The score of any point is then

$$g(\mathbf{x}) = \sum_{j=1}^{n} \alpha_j t_j \, \tilde{\mathbf{x}}_j^{\mathsf{T}} \tilde{\mathbf{x}},$$

so the algorithm can be run entirely on the counts α_j and on inner products between examples, without ever forming $\tilde{\mathbf{w}}$ (Algorithm 3). Examples that never caused a mistake have $\alpha_j = 0$ and drop out. Nothing is gained in this form for plain vectors, but it is the gateway to non-linear classification: replace every inner product $\tilde{\mathbf{x}}_j^{\mathsf{T}} \tilde{\mathbf{x}}$ by a *kernel* $k(\mathbf{x}_j, \mathbf{x})$ and the same few lines become the kernel perceptron of Note 10. The SVM has the same structure (Section 5.2).

Running time — perceptron (primal, averaged, dual)

Primal: each visit costs one inner product, $O(d)$, and an update another $O(d)$; one epoch is $O(nd)$ and E epochs $O(End)$. On separable data at most $(R/\gamma)^2$ updates happen in total, whatever n. Prediction $O(d)$; memory $O(d)$ beyond the data, which can be streamed from disk. *Averaged*: the same order, $O(nd)$ per epoch, with one extra vector of memory (the sum can even be maintained lazily, touching only the non-zero features of sparse inputs). *Pocket*: one error count per epoch costs another $O(nd)$, so the order stays $O(nd)$ per epoch; checking after every update instead would cost up to $O(n^2 d)$ per epoch. Memory: one extra vector. *Dual*: $O(n^2 d)$ to compute the Gram matrix and $O(n^2)$ memory to store it, then $O(n)$ per visit and $O(n^2)$ per epoch; prediction $O(n_\alpha d)$, with n_α the number of examples with $\alpha_j > 0$. The dual is worth its quadratic cost only when kernels are used (Note 10).

Algorithm 3 The perceptron in dual form.

Require: examples $(\mathbf{x}_i, t_i)$, epochs E
$K_{ij} \leftarrow \tilde{\mathbf{x}}_i^{\mathsf{T}} \tilde{\mathbf{x}}_j$ for all i, j ▷ Gram matrix, $n \times n$
$\alpha \leftarrow \mathbf{0} \in \mathbb{R}^n$
for epoch $= 1, \dots, E$ **do**
  mistakes $\leftarrow 0$
  for each i in random order **do**
    if $t_i \sum_j \alpha_j t_j K_{ji} \le 0$ **then** ▷ the same test as in Algorithm 2
      $\alpha_i \leftarrow \alpha_i + 1$, *mistakes* $\leftarrow$ *mistakes* $+ 1$
    end if
  end for
  if *mistakes* $= 0$ **then break**
  end if
end for
return α ▷ predict with sign $\big(\sum_j \alpha_j t_j \, \tilde{\mathbf{x}}_j^{\mathsf{T}} \tilde{\mathbf{x}}\big)$

4 Logistic regression

Logistic regression [6] fits a model of a probability, but it is used as a classifier. It is the discriminative counterpart of the Gaussian classifiers of Note 6: instead of modelling the classes and applying Bayes' rule, it models the class probability directly. In this section the labels are $t \in \{0, 1\}$.

4.1 The model

For two classes Bayes' rule can always be written as $P(t = 1 \mid \mathbf{x}) = p(\mathbf{x} \mid 1)P_1 / [p(\mathbf{x} \mid 1)P_1 + p(\mathbf{x} \mid 0)P_0] = 1/(1+e^{-a(\mathbf{x})})$, with the priors $P_c = P(t = c)$ and the log-odds $a(\mathbf{x}) = \log[p(\mathbf{x} \mid 1)P_1 / p(\mathbf{x} \mid 0)P_0]$. Under the LDA assumptions $a(\mathbf{x})$ is exactly linear (Note 6, Section 3). Logistic regression keeps this form and drops the assumptions: it postulates

$$P(t = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b) = \sigma(\tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}), \qquad \sigma(z) = \frac{1}{1 + e^{-z}},$$

so that the log-odds are linear in $\mathbf{x}$: solving $P = \sigma(a)$ for a gives $a = \operatorname{logit}(P) = \log[P/(1 - P)]$. The sigmoid is thus the inverse of the *logit link*, which makes logistic regression a *generalised linear model*: a linear predictor passed through a fixed link to the mean of the label. The logistic sigmoid maps $\mathbb{R}$ to $(0, 1)$, is a smooth version of a step function, and has two properties used below:

$$1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}} = \sigma(-z), \qquad \sigma'(z) = \frac{e^{-z}}{(1 + e^{-z})^2} = \sigma(z)\,\frac{e^{-z}}{1 + e^{-z}} = \sigma(z)\big(1 - \sigma(z)\big).$$

Since $\sigma(z) > \frac{1}{2}$ exactly when $z > 0$, predicting the more probable class is still $\operatorname{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x} + b)$: the boundary is a hyperplane. What logistic regression adds is a probability, and a loss that does not punish confident correct predictions. In neural terms it is a single neuron with a sigmoid activation: the sigmoid is a differentiable stand-in for the perceptron's step, and a rough model of how a biological neuron's firing rate saturates as its input grows (week 5 builds networks of such units).

4.2 Maximum likelihood and the gradient, step by step

A warning on notation. In Note 8 the letter y_i was the regression target; in this note (as in Bishop's book [3]) targets are always t_i, and y_i is the model's output. Write $y_i = \sigma(a_i)$ with $a_i = \tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i$ for

the predicted probability (the g of Note 4, Section 2). The probability of label t_i is $y_i^{t_i}(1-y_i)^{1-t_i}$, and the negative log-likelihood of independent labels (Note 2, Section 2) is the cross-entropy (Note 4, Section 2)

$$J(\tilde{\mathbf{w}}) = \sum_{i=1}^{n} J_i, \qquad J_i = -\big[t_i \log y_i + (1-t_i)\log(1-y_i)\big].$$

In terms of the ± 1 margin $z_i = (2t_i - 1)a_i$ this is $J_i = \log(1 + e^{-z_i})$, the logistic curve of Figure 2. Indeed, for $t_i = 1$, $J_i = -\log\sigma(a_i) = \log(1+e^{-a_i})$, and for $t_i = 0$, $J_i = -\log(1-\sigma(a_i)) = -\log\sigma(-a_i) = \log(1+e^{a_i})$; both read $\log(1+e^{-z_i})$, with $z_i = a_i$ and $z_i = -a_i$ respectively. The gradient follows from the chain rule $J_i \to y_i \to a_i \to \tilde{\mathbf{w}}$:

$$\frac{\partial J_i}{\partial y_i} = -\frac{t_i}{y_i} + \frac{1-t_i}{1-y_i} = \frac{y_i - t_i}{y_i(1-y_i)}, \qquad \frac{\partial y_i}{\partial a_i} = y_i(1-y_i), \qquad \frac{\partial a_i}{\partial \tilde{\mathbf{w}}} = \tilde{\mathbf{x}}_i.$$

The factor $y_i(1-y_i)$ cancels, and

$$\nabla J = \sum_{i=1}^{n} (y_i - t_i)\,\tilde{\mathbf{x}}_i = \tilde{X}^{\mathsf{T}}(\mathbf{y} - \mathbf{t}). \tag{4}$$

This has the same form as the least-squares gradient of Note 8 (with Note 8's target y_i renamed t_i), *prediction minus target, times input*, but now the prediction is a probability. Each example pushes $\tilde{\mathbf{w}}$ in proportion to how wrong its probability is: a confidently correct point ($y_i \approx t_i$) contributes almost nothing, which is why distant easy points do not pull the boundary.

> **Worked example — one gradient step**
>
> Six points on a line, $x = -2, -1, -1, 1, 1, 2$ with labels $t = 0, 0, 1, 0, 1, 1$; the classes overlap, since one positive sits at $x = -1$ and one negative at $x = 1$. Start from $\tilde{\mathbf{w}} = (b, w) = (0, 0)$: every $y_i = \sigma(0) = \frac{1}{2}$ and $J = 6\log 2 \approx 4.159$. The residuals $y_i - t_i$ are $\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, \frac{1}{2}, -\frac{1}{2}, -\frac{1}{2}$, so by (4)
>
> $$\frac{\partial J}{\partial b} = \textstyle\sum_i (y_i - t_i) = 0, \qquad \frac{\partial J}{\partial w} = \sum_i (y_i - t_i)x_i = -1 - \frac{1}{2} + \frac{1}{2} + \frac{1}{2} - \frac{1}{2} - 1 = -2.$$
>
> A gradient step with $\eta = 0.25$ gives $w = 0 - 0.25 \times (-2) = 0.5$, $b = 0$, and the loss drops to 3.523. The bias stays zero at every step here because the data are symmetric under $x \to -x$, $t \to 1 - t$; the boundary is at $x = 0$ and only the steepness of the sigmoid is being learned.

4.3 Training: gradient descent and Newton's method

The Hessian follows by differentiating (4) once more. In the term $(y_i - t_i)\,\tilde{\mathbf{x}}_i$ only y_i depends on $\tilde{\mathbf{w}}$, with $\partial y_i/\partial \tilde{\mathbf{w}} = y_i(1-y_i)\,\tilde{\mathbf{x}}_i$, so its derivative is the outer product $\tilde{\mathbf{x}}_i\,(\partial y_i/\partial \tilde{\mathbf{w}})^{\mathsf{T}} = y_i(1-y_i)\,\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^{\mathsf{T}}$, and

$$\mathsf{H} = \nabla^2 J = \sum_{i=1}^{n} y_i(1-y_i)\,\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^{\mathsf{T}} = \tilde{X}^{\mathsf{T}} S\,\tilde{X}, \qquad S = \operatorname{diag}\big(y_i(1-y_i)\big).$$

(We write the Hessian as H to keep H for the decision boundary.) For any vector $\mathbf{v}$, $\mathbf{v}^{\mathsf{T}}\mathsf{H}\mathbf{v} = \sum_i y_i(1-y_i)(\mathbf{v}^{\mathsf{T}}\tilde{\mathbf{x}}_i)^2 \geq 0$: J is convex (Note 7, Section 4), so every local minimum is global. But $\nabla J = \mathbf{0}$ is nonlinear in $\tilde{\mathbf{w}}$ and has no closed-form solution, so the minimum is found iteratively. Gradient descent (Algorithm 4) uses $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \eta\,\tilde{X}^{\mathsf{T}}(\mathbf{y} - \mathbf{t})$, or the same with mini-batches for large n (Note 7, Section 5). Newton's method (Note 7, Section 6) uses the curvature: $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \mathsf{H}^{-1}\nabla J$. Writing $\tilde{\mathbf{w}} = \mathsf{H}^{-1}\mathsf{H}\tilde{\mathbf{w}}$, inserting $SS^{-1} = I$ in the gradient and factoring out $\tilde{X}^{\mathsf{T}}S$,

$$\tilde{\mathbf{w}} - \mathsf{H}^{-1}\nabla J = \mathsf{H}^{-1}\big(\tilde{X}^{\mathsf{T}}S\tilde{X}\,\tilde{\mathbf{w}} - \tilde{X}^{\mathsf{T}}S\,S^{-1}(\mathbf{y} - \mathbf{t})\big) = \mathsf{H}^{-1}\tilde{X}^{\mathsf{T}}S\big(\tilde{X}\tilde{\mathbf{w}} - S^{-1}(\mathbf{y} - \mathbf{t})\big) = (\tilde{X}^{\mathsf{T}}S\tilde{X})^{-1}\tilde{X}^{\mathsf{T}}S\,\mathbf{u},$$

with the *working targets* $\mathbf{u} = \tilde{X}\tilde{\mathbf{w}} - S^{-1}(\mathbf{y} - \mathbf{t})$. Compare with *weighted* least squares: minimising $\sum_i s_i(\tilde{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}}_i - u_i)^2 = (\tilde{X}\tilde{\mathbf{w}} - \mathbf{u})^\mathsf{T} S (\tilde{X}\tilde{\mathbf{w}} - \mathbf{u})$ and setting the gradient to zero gives $\tilde{X}^\mathsf{T} S \tilde{X}\,\tilde{\mathbf{w}} = \tilde{X}^\mathsf{T} S\,\mathbf{u}$, the normal equations of Note 8 with row i weighted by s_i. So each Newton step is a weighted least-squares fit to the working targets $\mathbf{u}$ with weights $s_i = y_i(1-y_i)$, hence the name *iteratively reweighted least squares* (IRLS). Points whose probability is already close to 0 or 1 receive little weight. Methods between the two extremes, such as conjugate gradient and quasi-Newton methods like L-BFGS, are named in Note 7, Section 6, and work equally well here.

Worked example — Newton against gradient descent

For the six points above, at $\tilde{\mathbf{w}} = \mathbf{0}$ all weights are $\frac{1}{4}$, so $\mathsf{H} = \frac{1}{4}\sum_i \tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^\mathsf{T} = \frac{1}{4}\begin{pmatrix} 6 & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix} = \begin{pmatrix} 1.5 & 0 \\ 0 & 3 \end{pmatrix}$, and the first Newton step is $\tilde{\mathbf{w}} = -\mathsf{H}^{-1}\nabla J = -(0/1.5,\ -2/3) = (0,\ 0.667)$. Iterating,

iteration k	0	1	2	3	10
Newton: w, J	0, 4.159	0.667, 3.459	0.753, 3.450	0.756, 3.450	—
gradient descent ($\eta = 0.25$): w, J	0, 4.159	0.500, 3.523	0.646, 3.463	0.706, 3.453	0.756, 3.450

Newton reaches the minimum $w^* = 0.756$ to three digits in three steps; gradient descent needs about ten ($w_9 = 0.7557$, $w_{10} = 0.7560$). The fitted model says $P(t = 1 \mid x = 2) = \sigma(1.51) = 0.82$ and $P(t = 1 \mid x = 1) = 0.68$: the overlap of the classes near $x = \pm 1$ is reflected in moderate probabilities.

Algorithm 4 Logistic regression (gradient descent and Newton/IRLS) and softmax regression.

Require: $\tilde{X} \in \mathbb{R}^{n\times(d+1)}$, $\mathbf{t} \in \{0,1\}^n$, ridge $\lambda \geq 0$ (see below), step η (GD), tolerance ϵ

$\tilde{\mathbf{w}} \leftarrow \mathbf{0}$; $D \leftarrow \mathrm{diag}(0, 1, \ldots, 1)$ ▷ the bias is not penalised
repeat
  $\mathbf{y} \leftarrow \sigma(\tilde{X}\tilde{\mathbf{w}})$ ▷ element-wise
  $\mathbf{g} \leftarrow \tilde{X}^\mathsf{T}(\mathbf{y} - \mathbf{t}) + \lambda D\tilde{\mathbf{w}}$
  if gradient descent **then** $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \eta\,\mathbf{g}$
  else $S \leftarrow \mathrm{diag}(y_i(1-y_i))$; $\mathsf{H} \leftarrow \tilde{X}^\mathsf{T} S \tilde{X} + \lambda D$; $\tilde{\mathbf{w}} \leftarrow \tilde{\mathbf{w}} - \mathsf{H}^{-1}\mathbf{g}$ ▷ Newton: solve, do not invert
  end if
until $\|\mathbf{g}\| < \epsilon$
return $\tilde{\mathbf{w}}$ ▷ predict $\sigma(\tilde{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}})$, class $\mathbf{1}\{\tilde{\mathbf{w}}^\mathsf{T}\tilde{\mathbf{x}} > 0\}$

function SOFTMAX($\tilde{X}$, one-hot $T \in \{0,1\}^{n\times m}$, λ, η, ϵ) ▷ m classes, Section 4.4
  $\tilde{W} \leftarrow \mathbf{0} \in \mathbb{R}^{(d+1)\times m}$ ▷ column c is $\tilde{\mathbf{w}}_c$
  repeat
    $A \leftarrow \tilde{X}\tilde{W}$; $Y_{ic} \leftarrow e^{A_{ic}} / \sum_k e^{A_{ik}}$ ▷ row-wise softmax; subtract $\max_k A_{ik}$ first
    $G \leftarrow \tilde{X}^\mathsf{T}(Y - T) + \lambda D\tilde{W}$; $\tilde{W} \leftarrow \tilde{W} - \eta\,G$
  until $\|G\| < \epsilon$
  return $\tilde{W}$ ▷ predict $\arg\max_c (\tilde{W}^\mathsf{T}\tilde{\mathbf{x}})_c$
end function

Separable data and regularisation. If the classes are separable, every margin can be made positive, and multiplying $\tilde{\mathbf{w}}$ by any $c > 1$ then lowers every term $\log(1 + e^{-z_i})$. The loss has no minimiser: $\|\tilde{\mathbf{w}}\|$ grows without bound and the sigmoid tends to a step, with probabilities of exactly 0 and 1 —

overconfidence. Adding a ridge penalty $\frac{\lambda}{2}\|\mathbf{w}\|^2$ (Note 8, Section 4), as in Algorithm 4, restores a unique finite solution; λ is a hyperparameter chosen by validation (Note 4).

4.4 More than two classes: softmax regression

With m classes, give each class a score $a_c = \tilde{\mathbf{w}}_c^{\mathsf{T}}\tilde{\mathbf{x}}$ and turn the scores into probabilities with the *softmax*

$$y_c = P(t = c \mid \mathbf{x}) = \frac{e^{a_c}}{\sum_{k=1}^{m} e^{a_k}}.$$

With a one-hot target vector $\mathbf{t}$ the cross-entropy of one example is $J = -\sum_c t_c \log y_c = -a_{\text{true}} + \log \sum_k e^{a_k}$. Differentiating with respect to the score a_k,

$$\frac{\partial J}{\partial a_k} = -t_k + \frac{e^{a_k}}{\sum_{k'} e^{a_{k'}}} = y_k - t_k, \qquad \nabla_{\tilde{\mathbf{w}}_k} J = (y_k - t_k)\,\tilde{\mathbf{x}},$$

the same "prediction minus target" form as before. A gradient step raises the score of the true class and lowers every other score in proportion to the probability it wrongly received. For $m = 2$ the softmax reduces to the sigmoid of the score difference, $y_2 = e^{a_2}/(e^{a_1} + e^{a_2}) = \sigma(a_2 - a_1)$, so softmax regression generalises logistic regression; since adding the same vector to every $\tilde{\mathbf{w}}_c$ leaves all probabilities unchanged, one class can be fixed at zero, or a ridge penalty used.

Worked example — softmax and its gradient

Scores $\mathbf{a} = (2, 1, 0)$ give $e^{\mathbf{a}} = (7.389, 2.718, 1)$, sum 11.107, so $\mathbf{y} = (0.665, 0.245, 0.090)$. If the true class is the second, the loss is $-\log 0.245 = 1.41$ and $\mathbf{y} - \mathbf{t} = (0.665, -0.755, 0.090)$: the step mostly lowers class 1 and raises class 2; class 3, already improbable, is barely touched. Concretely, if these scores come from the input $\tilde{\mathbf{x}} = (1, 2)$ (bias feature and one coordinate), a step with $\eta = 0.2$ changes each column by $\Delta\tilde{\mathbf{w}}_k = -\eta\,(y_k - t_k)\,\tilde{\mathbf{x}}$ and hence each score by $-\eta\,(y_k - t_k)\,\|\tilde{\mathbf{x}}\|^2 = -(y_k - t_k)$: the new scores are $(1.335, 1.755, -0.090)$, and the true class now has the highest.

Running time — logistic and softmax regression

Gradient descent: each iteration computes $\tilde{X}\tilde{\mathbf{w}}$ and $\tilde{X}^{\mathsf{T}}(\mathbf{y} - \mathbf{t})$, two matrix–vector products, $O(nd)$; T iterations cost $O(Tnd)$, with T from tens to thousands depending on the conditioning (Note 7). Mini-batch SGD with mini-batch size n' costs $O(n'd)$ per step and suits data that do not fit in memory. *Newton/IRLS*: forming $\tilde{X}^{\mathsf{T}}S\tilde{X}$ costs $O(nd^2)$ and solving the system $O(d^3)$ per iteration; typically 5 to 10 iterations suffice. Newton is the method of choice up to d of a few thousand; beyond, gradient methods or L-BFGS (a quasi-Newton method that approximates the inverse Hessian from the last few gradients, Note 7). *Softmax*: $O(ndm)$ per gradient iteration (the product $\tilde{X}\tilde{W}$), and $O(nm)$ for the softmax itself. Prediction is $O(d)$ ($O(dm)$ for softmax), and all of it vectorises well on GPUs. *Memory* beyond the data: $O(d)$ for gradient descent ($O(dm)$ for softmax) plus $O(n)$ for $\mathbf{y}$, or $O(n'd)$ for a mini-batch when the data are streamed; Newton needs $O(d^2)$ for the Hessian, which is what rules it out for very large d.

Modern connection — every neural classifier ends in logistic regression

The last layer of an image classifier or a language model is a linear map followed by a softmax: multiclass logistic regression on features computed by the layers below, trained with exactly the gradient $\mathbf{y} - \mathbf{t}$ derived above. Conversely, training only a logistic regression on the frozen features of a pre-trained network — a *linear probe* — is a standard way to measure what those features contain [1]; CLIP, for instance, was evaluated with linear probes on 27 datasets [14]. Everything in this section therefore applies on top of the most modern representations.

5 Support vector machines

The perceptron stops at any separating hyperplane; Table 1 ended with a line that nearly touches two points. Among all separating hyperplanes, the *maximum-margin* one stays as far as possible from the closest training points, so that small perturbations of the data — noise, a slightly different test point — do not cross it. Support vector machines compute this hyperplane. The maximum-margin hyperplane goes back to Vapnik and Lerner's "generalized portrait" of 1963 [17]; the modern algorithm is that of Boser, Guyon and Vapnik [4], in the soft-margin form of Cortes and Vapnik [5]. Labels are again $t \in \{-1, +1\}$.

5.1 Margins and the hard-margin SVM

The *functional margin* of an example is $t_i\, g(\mathbf{x}_i)$: positive if it is correctly classified, large if confidently so. Its *geometric margin* $t_i\, g(\mathbf{x}_i)/\,\|\mathbf{w}\|$ is its signed distance to the boundary (1); the margin of the classifier is the smallest of these. Rescaling $(\mathbf{w}, b)$ by $c > 0$ multiplies functional margins by c but leaves geometric margins unchanged. We may therefore fix the scale so that the closest points have functional margin exactly one, $\min_i t_i\, g(\mathbf{x}_i) = 1$. These points lie on the two hyperplanes $g(\mathbf{x}) = \pm 1$, which by (1) are at distance $1/\,\|\mathbf{w}\|$ on either side of H. To keep the words apart: the *margin* of the classifier is $1/\,\|\mathbf{w}\|$, the distance from H to the closest points, and the empty band between the two hyperplanes has *width* $2/\,\|\mathbf{w}\|$.

The hard-margin SVM. Maximising the margin $\max_{\mathbf{w},b} \min_i t_i\, g(\mathbf{x}_i)/\,\|\mathbf{w}\|$ thus becomes, after fixing the scale, maximising $1/\,\|\mathbf{w}\|$ subject to all functional margins being at least one — equivalently minimising $\frac{1}{2}\,\|\mathbf{w}\|^2$:

$$\min_{\mathbf{w},b}\ \tfrac{1}{2}\,\|\mathbf{w}\|^2 \quad \text{subject to} \quad t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n. \tag{5}$$

Replacing the normalisation $\min_i t_i\, g(\mathbf{x}_i) = 1$ by the constraints $t_i\, g(\mathbf{x}_i) \geq 1$ for all i changes nothing at the optimum: if every constraint held strictly, $t_i\, g(\mathbf{x}_i) > 1$, we could shrink $(\mathbf{w}, b)$ by a factor slightly below one, stay feasible and reduce $\|\mathbf{w}\|$; so at the optimum at least one constraint is active, which is the normalisation we wanted. Fixing the scale by $\|\mathbf{w}\| = 1$ instead gives a third equivalent problem, maximising the smallest functional margin over unit vectors $\mathbf{w}$. All three formulations give the same hyperplane H and differ only in the length of $\mathbf{w}$.

This is a *quadratic programme*: a convex quadratic objective with linear constraints (Note 7, Section 6), with a unique solution if the data are separable.

5.2 Lagrange multipliers, the dual and support vectors

The constraints of (5) are inequalities, not the equalities of the Lagrange multipliers met in calculus, so the method needs one more idea. Attach a multiplier $\alpha_i \geq 0$ to each constraint (we reuse the letter α of the dual perceptron (3) on purpose: the two play the same role) and form the Lagrangian

$$L(\mathbf{w}, b, \alpha) = \tfrac{1}{2}\,\|\mathbf{w}\|^2 - \sum_{i=1}^{n} \alpha_i \big[t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) - 1\big].$$

For a feasible $(\mathbf{w}, b)$ every bracket is ≥ 0, so with $\alpha_i \geq 0$ we have $L \leq \frac{1}{2}\,\|\mathbf{w}\|^2$. Hence, for every $\alpha \geq 0$, the minimum of L over *all* $(\mathbf{w}, b)$ is a lower bound on the optimal value of (5); the *dual problem* asks for the largest such bound, which is why it is a maximisation over α. For a convex quadratic

programme like this one the best bound is attained: dual and primal optima are equal. The minimum over $(\mathbf{w}, b)$ is where the derivatives vanish,

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i t_i \mathbf{x}_i = \mathbf{0} \;\Rightarrow\; \mathbf{w} = \sum_{i=1}^{n} \alpha_i t_i \, \mathbf{x}_i, \qquad \frac{\partial L}{\partial b} = -\sum_i \alpha_i t_i = 0.$$

To substitute back, use $\sum_i \alpha_i t_i \, \mathbf{w}^{\mathsf{T}}\mathbf{x}_i = \mathbf{w}^{\mathsf{T}}\big(\sum_i \alpha_i t_i \mathbf{x}_i\big) = \mathbf{w}^{\mathsf{T}}\mathbf{w}$ and $\sum_i \alpha_i t_i \, b = 0$:

$$L = \tfrac{1}{2}\mathbf{w}^{\mathsf{T}}\mathbf{w} - \mathbf{w}^{\mathsf{T}}\mathbf{w} - 0 + \sum_i \alpha_i = \sum_i \alpha_i - \tfrac{1}{2}\Big\| \sum_i \alpha_i t_i \mathbf{x}_i \Big\|^2,$$

and expanding the squared norm gives the *dual problem*

$$\max_{\alpha} \; \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j t_i t_j \, \mathbf{x}_i^{\mathsf{T}} \mathbf{x}_j \quad \text{subject to} \quad \alpha_i \geq 0, \quad \sum_i \alpha_i t_i = 0. \tag{6}$$

At the optimum, the Karush–Kuhn–Tucker (KKT) conditions add *complementary slackness*,

$$\alpha_i \big[t_i \, g(\mathbf{x}_i) - 1 \big] = 0 \qquad \text{for every } i.$$

It follows from the equality of the two optima: at the solution $L = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_i \alpha_i [t_i \, g(\mathbf{x}_i) - 1]$ must equal $\frac{1}{2}\|\mathbf{w}\|^2$, so the sum vanishes, and since each of its terms is ≥ 0, each term vanishes. Either an example lies exactly on the margin ($t_i \, g(\mathbf{x}_i) = 1$) or its multiplier vanishes: a constraint that is not tight could be removed without changing the solution, so it exerts no "force" on it. The examples with $\alpha_i > 0$ are the *support vectors*: $\mathbf{w}$ is a combination of them alone, and moving any other point, without crossing the margin, changes nothing. The bias follows from any support vector s: multiplying $t_s(\mathbf{w}^{\mathsf{T}}\mathbf{x}_s + b) = 1$ by t_s gives $b = t_s - \mathbf{w}^{\mathsf{T}}\mathbf{x}_s$. Like the dual perceptron (3), the dual SVM needs the data only through inner products $\mathbf{x}_i^{\mathsf{T}}\mathbf{x}_j$, which is what makes the kernel SVM of Note 10, Section 7, possible. In practice the α_i are found numerically: the dual-coordinate-ascent algorithm given there also works with the linear kernel $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^{\mathsf{T}}\mathbf{z}$, for the slightly modified problem in which the bias is absorbed and penalised, so that the constraint $\sum_i \alpha_i t_i = 0$ drops out (see also the running-time box below). Here we find them by hand, by guessing the support vectors and checking.

Worked example — the maximum-margin line by hand

Take the four points of the perceptron trace: positives $\mathbf{x}_1 = (1, 2)$, $\mathbf{x}_3 = (3, 1)$; negatives $\mathbf{x}_2 = (0, 0)$, $\mathbf{x}_4 = (2, 0)$.

0. A first guess that fails. The closest pair of opposite points is $\mathbf{x}_3, \mathbf{x}_4$. With only them on the margin, $\mathbf{w}$ is parallel to $\mathbf{x}_3 - \mathbf{x}_4 = (1, 1)$, say $\mathbf{w} = k(1, 1)$, and $g(\mathbf{x}_3) = 4k + b = 1$, $g(\mathbf{x}_4) = 2k + b = -1$ give $k = 1$, $b = -3$. But then $g(\mathbf{x}_1) = 1 + 2 - 3 = 0 < 1$: the constraint of $\mathbf{x}_1$ is violated, so $\mathbf{x}_1$ must be on the margin too.

1. Guess the support vectors. The closest point of the negative class to the segment joining the positives is $\mathbf{x}_4$; we guess that $\mathbf{x}_1$ and $\mathbf{x}_3$ lie on $g = +1$ and $\mathbf{x}_4$ on $g = -1$.

2. Solve. Since $\mathbf{x}_1$ and $\mathbf{x}_3$ are both on $g = +1$, $\mathbf{w}$ is perpendicular to $\mathbf{x}_3 - \mathbf{x}_1 = (2, -1)$, so $\mathbf{w} = k(1, 2)$. The conditions $g(\mathbf{x}_1) = 5k + b = 1$ and $g(\mathbf{x}_4) = 2k + b = -1$ give $k = \frac{2}{3}$, $b = -\frac{7}{3}$:

$$\mathbf{w} = \big(\tfrac{2}{3}, \tfrac{4}{3}\big), \qquad b = -\tfrac{7}{3}, \qquad g(\mathbf{x}) = \tfrac{1}{3}(2x_1 + 4x_2 - 7).$$

3. Check feasibility. $g(\mathbf{x}_3) = \frac{1}{3}(6 + 4 - 7) = 1$ and $t_2 \, g(\mathbf{x}_2) = \frac{7}{3} \geq 1$: all constraints hold, and $\mathbf{x}_2$ is not on the margin.

4. Check optimality. Solve $\mathbf{w} = \sum_i \alpha_i t_i \mathbf{x}_i$ and $\sum_i \alpha_i t_i = 0$ with $\alpha_2 = 0$: $\alpha_1 + \alpha_3 = \alpha_4$ and

$\alpha_1(1,2) + \alpha_3(3,1) - \alpha_4(2,0) = (\frac{2}{3}, \frac{4}{3})$ give $\alpha_1 = \frac{2}{9}$, $\alpha_3 = \frac{8}{9}$, $\alpha_4 = \frac{10}{9}$. All are positive, so the KKT conditions hold and the guess was right; a negative α would have told us to drop that point from the guess.

Result. $\|\mathbf{w}\| = \frac{2\sqrt{5}}{3}$, the margin is $1/\|\mathbf{w}\| \approx 0.67$ and the band width $2/\|\mathbf{w}\| = 3/\sqrt{5} \approx 1.34$; the dual objective $\sum \alpha_i - \frac{1}{2}\|\mathbf{w}\|^2 = \frac{20}{9} - \frac{10}{9}$ equals the primal $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{10}{9}$. The perceptron's line $x_2 = 1/3$ kept only $1/3$ from the nearest points; the SVM keeps 0.67 on each side (Figure 4).

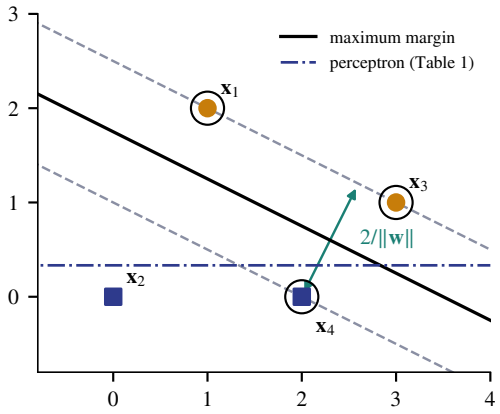


Figure 4. The maximum-margin line for the four points of Table 1. The dashed lines are $g(\mathbf{x}) = \pm 1$; the band between them has width $2/\|\mathbf{w}\| = 3/\sqrt{5}$. The three circled points are the support vectors; $\mathbf{x}_2$ could move anywhere on or below the lower dashed line ($2x_1 + 4x_2 \le 4$) without changing the solution. The dash-dotted line is where the perceptron stopped.

5.3 Soft margin and the hinge loss

For data that are not separable (5) has no solution. *Slack* variables $\xi_i \ge 0$ allow violations at a price:

$$\min_{\mathbf{w},b,\xi} \ \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n}\xi_i \quad \text{subject to} \quad t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b) \ge 1 - \xi_i, \quad \xi_i \ge 0.$$

An example with $0 < \xi_i < 1$ is inside the margin but correct, one with $\xi_i = 1$ lies on the boundary, and one with $\xi_i > 1$ is misclassified. For fixed $(\mathbf{w}, b)$ the best slack is the smallest allowed one, $\xi_i = \max\big(0, 1 - t_i\, g(\mathbf{x}_i)\big)$. Substituting it and dividing by C gives an unconstrained problem,

$$\min_{\mathbf{w},b} \ \sum_{i=1}^{n} \max\big(0,\ 1 - t_i(\mathbf{w}^{\mathsf{T}}\mathbf{x}_i + b)\big) + \frac{1}{2C}\|\mathbf{w}\|^2, \tag{7}$$

the hinge loss of Note 4 plus a ridge penalty: the SVM is regularised empirical risk minimisation, like everything else in this course. The hyperparameter C trades margin against training errors: small C means strong regularisation, a wide margin and many support vectors; large C fits the training data harder; $C \to \infty$ recovers the hard margin. In the dual, the only change is the box constraint $0 \le \alpha_i \le C$. Support vectors are now all points with $t_i\, g(\mathbf{x}_i) \le 1$: those on the margin ($0 \le \alpha_i \le C$) and those inside it or misclassified ($\alpha_i = C$), however far on the wrong side. The bias is computed from a support vector with $0 < \alpha_s < C$, which lies exactly on the margin. Soft-margin SVMs, linear or kernelised, were among the most widely used classifiers of the 2000s. They also extend to regression, with an ε-insensitive loss that ignores errors smaller than ε (support vector regression, Note 10, Section 8).

Worked example — slacks and the role of C

Keep the four points of the hand-solved example and add $\mathbf{x}_a = (1.5, 1.5)$ with $t_a = +1$ and $\mathbf{x}_b = (0, 3)$ with $t_b = -1$. For the line found there, $g(\mathbf{x}) = \frac{1}{3}(2x_1 + 4x_2 - 7)$, the functional margins $t_i\, g(\mathbf{x}_i)$ and slacks $\xi_i = \max(0, 1 - t_i\, g(\mathbf{x}_i))$ are

point	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_4$	$\mathbf{x}_a$	$\mathbf{x}_b$
$t_i\, g(\mathbf{x}_i)$	1	$\frac{7}{3}$	1	1	$\frac{2}{3}$	$-\frac{5}{3}$
ξ_i	0	0	0	0	$\frac{1}{3}$	$\frac{8}{3}$
status	on margin	beyond	on margin	on margin	inside, correct	misclassified

so $\sum_i \xi_i = 3$ and, with $\frac{1}{2}\|\mathbf{w}\|^2 = \frac{10}{9}$, the objective is $\frac{10}{9} + 3C$. Halving $(\mathbf{w}, b)$ keeps the same line but doubles the band: $\frac{1}{2}\|\mathbf{w}\|^2$ drops to $\frac{5}{18}$, while all margins halve, the slacks become $\frac{1}{2}, 0, \frac{1}{2}, \frac{1}{2}, \frac{2}{3}, \frac{11}{6}$ and their sum grows to 4, giving $\frac{5}{18} + 4C$. The wide band is better when $C < \frac{5}{6}$ (at $C = 0.1$: 0.68 against 1.41) and the narrow one when $C > \frac{5}{6}$ (at $C = 10$: 31.1 against 40.3): small C buys a wide margin at the price of slack, large C does the opposite.

5.4 Training by subgradient descent

The objective (7) is convex but has a kink where $t_i\, g(\mathbf{x}_i) = 1$, where no gradient exists. A *subgradient* of a convex function J at $\mathbf{w}$ is any vector $\mathbf{v}$ with $J(\mathbf{w}') \geq J(\mathbf{w}) + \mathbf{v}^{\mathsf{T}}(\mathbf{w}' - \mathbf{w})$ for all $\mathbf{w}'$: the plane with slope $\mathbf{v}$ touches the graph at $\mathbf{w}$ and stays below it. Where J is differentiable the only subgradient is the gradient; at the kink of the hinge any convex combination of the gradients of the two pieces qualifies, and we pick the flat piece's. Gradient descent with subgradients still converges if the step size decreases. To keep the algorithm short we use the bias trick, $\tilde{\mathbf{w}} = (b, \mathbf{w})$ and $\tilde{\mathbf{x}}_i = (1, \mathbf{x}_i)$, and write the objective as $\frac{\lambda}{2}\|\tilde{\mathbf{w}}\|^2 + \frac{1}{n}\sum_i \max(0, 1 - t_i\, \tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i)$ with $\lambda = 1/(nC)$. This also penalises the bias slightly, a simple option discussed in the Pegasos paper; with a large separate step on an unpenalised bias the iterates drift and converge very slowly. A subgradient of the term of example i is

$$\lambda\tilde{\mathbf{w}} - \mathbf{1}\{t_i\, \tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i < 1\}\, t_i\tilde{\mathbf{x}}_i.$$

Points beyond the margin contribute only the shrinkage $\lambda\tilde{\mathbf{w}}$; a point inside the margin or misclassified also pulls $\tilde{\mathbf{w}}$ towards $t_i\tilde{\mathbf{x}}_i$ — a perceptron update that fires already when $t_i\, g(\mathbf{x}_i) < 1$ instead of ≤ 0. Algorithm 5 is the stochastic version, known as Pegasos [16], with the step $\eta_k = 1/(\lambda k)$.

Algorithm 5 Soft-margin linear SVM by stochastic subgradient descent (Pegasos-style).

Require: augmented examples $(\tilde{\mathbf{x}}_i, t_i)$, $\tilde{\mathbf{x}}_i = (1, \mathbf{x}_i)$, $t_i \in \{-1, +1\}$; $\lambda = 1/(nC)$; number of steps T

$\tilde{\mathbf{w}} \leftarrow \mathbf{0}$ ▷ $\tilde{\mathbf{w}} = (b, \mathbf{w})$: the bias is shrunk like the weights
for $k = 1, \ldots, T$ **do**
  pick i uniformly at random; $\eta \leftarrow 1/(\lambda k)$
  if $t_i\, \tilde{\mathbf{w}}^{\mathsf{T}}\tilde{\mathbf{x}}_i < 1$ **then** ▷ inside the margin or wrong
    $\tilde{\mathbf{w}} \leftarrow (1 - \eta\lambda)\, \tilde{\mathbf{w}} + \eta\, t_i\tilde{\mathbf{x}}_i$
  else
    $\tilde{\mathbf{w}} \leftarrow (1 - \eta\lambda)\, \tilde{\mathbf{w}}$ ▷ only the ridge shrinkage
  end if
end for
return $\tilde{\mathbf{w}}$ ▷ in practice, often the average of the last iterates

At $k = 1$ the shrinkage factor $1 - \eta\lambda$ is zero, so the first step simply sets $\tilde{\mathbf{w}} = \eta\, t_i\tilde{\mathbf{x}}_i$ whatever the example; this is expected, not an error. (The basic Pegasos algorithm has no bias term; absorbing it with the bias trick is the simplest way to add one.)

Worked example — two Pegasos steps

On the four points of the perceptron trace take $\lambda = 1$ (large, to keep the numbers simple). Step $k = 1$, $\eta = 1$, visits $\mathbf{x}_1$, $\tilde{\mathbf{x}}_1 = (1, 1, 2)$: its margin is $0 < 1$, so $\tilde{\mathbf{w}} = (1 - 1)\,\mathbf{0} + 1 \cdot \tilde{\mathbf{x}}_1 = (1, 1, 2)$, i.e. $b = 1$, $\mathbf{w} = (1, 2)$. Step $k = 2$, $\eta = \frac{1}{2}$, visits $\mathbf{x}_4$, $\tilde{\mathbf{x}}_4 = (1, 2, 0)$, $t_4 = -1$: its margin is $-(1 + 2) = -3 < 1$, so $\tilde{\mathbf{w}} = \frac{1}{2}(1, 1, 2) - \frac{1}{2}(1, 2, 0) = (0, -0.5, 1)$, i.e. $b = 0$, $\mathbf{w} = (-0.5, 1)$. The first factor $(1 - \eta\lambda)$ shrinks the old weights, the second term is a perceptron-like push; with the steps shrinking like $1/k$, later examples move $\tilde{\mathbf{w}}$ less and less.

5.5 More than two classes

Binary classifiers extend in two standard ways (Algorithm 6). *One-versus-rest* (OvR) trains m classifiers, each separating one class from all the others, and predicts the class with the highest score. *One-versus-one* (OvO) trains $m(m-1)/2$ classifiers, one for each pair of classes on the data of those two classes only, and lets them vote. With raw yes/no answers there are *ambiguous regions*, where several or no classifiers claim a point. Taking the highest OvR score removes them (up to exact ties). OvO voting shrinks them but can still tie: "1 beats 2, 2 beats 3, 3 beats 1" gives every class one vote; such ties are usually broken by summing the pairwise scores. Figure 5 shows the regions for three classes. Softmax regression (Section 4) needs neither scheme: it is multiclass from the start. The same is true of the multiclass SVM of Crammer and Singer [7], which trains all m score functions jointly by minimising the multiclass hinge loss $\max\big(0, 1 + \max_{k \neq t} g_k(\mathbf{x}) - g_t(\mathbf{x})\big)$ of Note 4, Section 2, plus a ridge penalty.

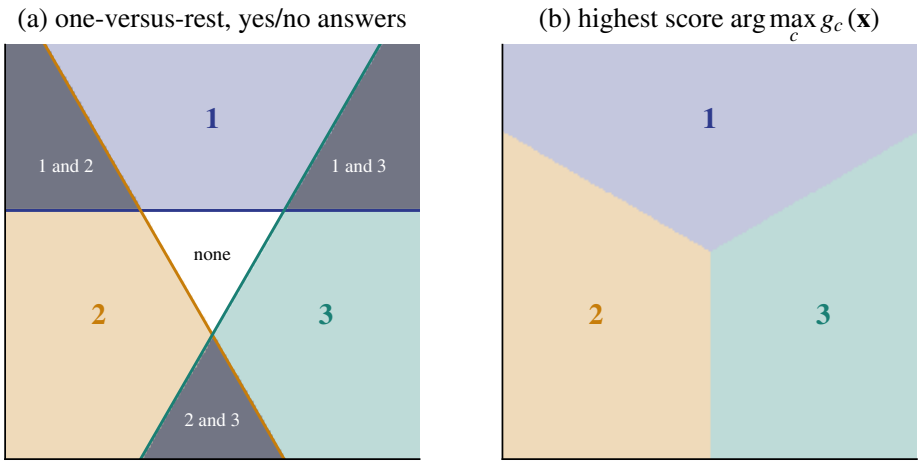


Figure 5. Three classes, one-versus-rest with linear scores g_c. (a) Read as yes/no answers, the three lines $g_c = 0$ leave dark regions claimed by two classes and a central triangle claimed by none. (b) Taking the highest score assigns every point to exactly one class; the boundaries are where two scores are equal.

Worked example — the two schemes can disagree

With $m = 3$, a point gets OvR scores $g_1 = 0.8$, $g_2 = 0.3$, $g_3 = -1.1$: as yes/no answers classifiers 1 and 2 both claim it (an ambiguous region), but the highest score picks class 1. The OvO scores $g_{12} = +0.2$ (1 beats 2), $g_{13} = -0.4$ (3 beats 1), $g_{23} = -0.5$ (3 beats 2) give class 3 two votes: the schemes, trained on different data, need not agree.

Algorithm 6 One-versus-rest and one-versus-one.

Require: data $\{(\mathbf{x}_i, t_i)\}$ with classes $1, \dots, m$; a binary learner BIN returning a score function

function TRAINOVR(data) ▷ predict $\arg\max_c g_c(\mathbf{x})$
  for $c = 1, \dots, m$ **do**
    $g_c \leftarrow$ BIN(all $\mathbf{x}_i$, label $+1$ if $t_i = c$, else -1)
  end for
  return $\{g_c\}$
end function
function TRAINOVO(data)
  for each pair $c < c'$ **do**
    $g_{cc'} \leftarrow$ BIN(only the $\mathbf{x}_i$ of class c (label $+1$) and c' (label -1))
  end for
  return $\{g_{cc'}\}$
end function
function PREDICTOVO($\mathbf{x}$)
  $v_c \leftarrow$ number of pairs won by c ▷ $g_{cc'}(\mathbf{x}) > 0$: a win for c, else for c'
  return $\arg\max_c v_c$ ▷ ties: largest sum of pairwise scores
end function

Running time — linear SVM and multiclass schemes

Subgradient SVM (Pegasos): $O(d)$ per step, independent of n. For a fixed regularisation strength λ, reaching accuracy ϵ on the objective takes about $1/(\lambda\epsilon)$ steps up to logarithmic factors [16], a cost $O(d/(\lambda\epsilon))$ that does not depend on n; with $\lambda = 1/(nC)$ at fixed C it is $O(dnC/\epsilon)$, linear in n and with no $n \times n$ matrix. Memory $O(d)$; prediction $O(d)$. *Dual solvers*: generic quadratic-programming solvers form the $n \times n$ matrix of inner products, $O(n^2 d)$ time and $O(n^2)$ memory; decomposition methods such as SMO compute its rows on demand with a cache and take between $O(n^2)$ and $O(n^3)$ time in practice — fine for n up to some 10^4–10^5, impractical for millions. Kernel SVMs (Note 10) must work this way, but a *linear* SVM need not: dual coordinate descent keeps $\mathbf{w} = \sum_i \alpha_i t_i \mathbf{x}_i$ up to date at $O(nd)$ per epoch with no Gram matrix (the method behind the LIBLINEAR library). *OvR*: m binary trainings on all n points, prediction $O(md)$, memory $O(md)$. *OvO*: $m(m-1)/2$ trainings on about $2n/m$ points each, prediction $O(m^2 d)$, memory $O(m^2 d)$. For a learner linear in n, both total about mn example visits; for a learner quadratic in n, such as a dual QP solver, OvO's smaller problems make it the cheaper scheme.

6 Limits of linear classifiers

Whatever the algorithm, a linear classifier draws a hyperplane, and some problems have no good hyperplane at all. The classic example is the XOR pattern: the points $(0, 0)$ and $(1, 1)$, with equal coordinates, in class $t = +1$, and $(0, 1)$ and $(1, 0)$ in class $t = -1$ (the same convention as Note 10, Section 1). Suppose $g(\mathbf{x}) = w_1 x_1 + w_2 x_2 + b$ separated them, positive on the first pair and negative on the second:

$$b > 0, \qquad w_1 + w_2 + b > 0, \qquad w_2 + b < 0, \qquad w_1 + b < 0.$$

Adding the first two inequalities gives $w_1 + w_2 + 2b > 0$, adding the last two gives $w_1 + w_2 + 2b < 0$: a contradiction. No line separates XOR, and no amount of training — perceptron, logistic or SVM — will change that; the perceptron simply never stops. Minsky and Papert's 1969 analysis of what single-layer perceptrons cannot compute (XOR is the simplest case of their parity results) is often credited with contributing to a decline of neural-network research in the 1970s [12]. Real data show the same defect in less obvious forms — a signal region surrounded by background, a class split into two islands, a curved boundary: the capacity of the hyperplane family (Note 4, Section 5) is too small

and the classifier underfits, however much data it is given.

The remedy keeps the losses, the optimisation and the margins, and changes the space in which the hyperplane is drawn: adding a single product feature makes XOR separable (Note 10, Section 1), and Question B7 asks for another such feature. Such features can be chosen explicitly (from physics, or as polynomials), implicitly through a kernel that replaces the inner products of the dual forms of Sections 3.6 and 5.2, or learned with the classifier, as in a neural network. Explicit feature maps and the kernel trick are the subject of Note 10; learned features are where week 5 begins.

In physics — feature engineering, the physicist's way

The invariant mass of a pair of photons, $m_{\gamma\gamma}^2 = 2E_1E_2(1-\cos\theta_{12})$, is a non-linear function of the measured energies and angles. In it, the signal of a Higgs boson decaying to two photons becomes a narrow peak on a smooth background, so a mass window keeps most of the signal while removing most of the background — something no single hyperplane in the raw energies and angles could do. Physics has always supplied such features. When deep networks were given only low-level detector quantities in benchmark searches for new particles, they matched or exceeded the performance obtained with the best hand-engineered variables, having learned equivalent features themselves [2].

Summary A linear classifier predicts $\text{sign}(\mathbf{w}^\mathsf{T}\mathbf{x}+b)$; its boundary is a hyperplane normal to $\mathbf{w}$, $g(\mathbf{x})/\|\mathbf{w}\|$ is a signed distance, and $t\,g(\mathbf{x})$ is the margin of an example. All the algorithms of this note learn hyperplanes, chosen by different losses of the margin; least squares is pulled by distant easy points. The perceptron adds misclassified examples to $\mathbf{w}$, each update raising that example's margin by $\|\mathbf{x}_i\|^2+1$; it converges after at most $(R/\gamma)^2$ mistakes on separable data, stops at any separator, and in dual form needs only inner products. Logistic regression gives probabilities, with gradient $\sum(y_i-t_i)\check{\mathbf{x}}_i$ and a convex loss minimised by gradient descent or Newton/IRLS; softmax extends it to m classes. The SVM maximises the margin $1/\|\mathbf{w}\|$ (band width $2/\|\mathbf{w}\|$), depends only on its support vectors, and in soft-margin form is hinge loss plus ridge. No hyperplane solves XOR: non-linear features are needed (Note 10).

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. What does it mean geometrically that $\mathbf{w}$ is normal to the decision boundary? What happens to the boundary when b is increased with $\mathbf{w}$ fixed, and what happens to the classifier when both $\mathbf{w}$ and b are multiplied by 3? By -1?

A2. Nearest centroid and LDA both give linear boundaries. What information about the classes does LDA use that nearest centroid ignores, and when do the two coincide?

A3. Explain, using the shape of the squared loss as a function of the margin, why a cluster of far-away but correctly classified points can make the least-squares classifier worse. Why are logistic regression and the SVM much less affected?

A4. Why can the training error rate not be minimised by gradient descent? What is a surrogate loss, and which surrogate does each of the perceptron, logistic regression and the SVM use?

A5. True or false, with a justification: "after a perceptron update on example i, example i is always classified correctly." What does always hold after the update?

A6. Describe in words and with a sketch what a perceptron update does to $\mathbf{w}$ and to the decision boundary when a positive example is misclassified. Why can an update spoil a point that was previously correct?

A7. On separable data the perceptron stops. Is the hyperplane it returns unique? Does it depend on the order in which the examples are visited? Why might it generalise poorly?

A8. What happens when the perceptron is run on non-separable data? Describe two remedies and what each returns.

A9. In the dual form of the perceptron, what does α_i count, which examples have $\alpha_i = 0$, and why is this form the starting point of kernel methods?

A10. Logistic regression has "regression" in its name. In what sense does it regress, and why is it nevertheless a linear classifier?

A11. Why do the weights of an unregularised logistic regression grow without bound on separable data? What is wrong with the resulting probabilities, and how is it fixed?

A12. Compare gradient descent and Newton's method for logistic regression: cost per iteration, number of iterations, and when you would choose each.

A13. What are support vectors? Explain why the soft-margin SVM solution does not change when a correctly classified point outside the margin is moved, as long as it stays outside the margin. Which group of points in Figure 1 (think of the distant amber cluster) has no influence on the linear SVM, and why does the same group change the least-squares line?

A14. What is the role of C in the soft-margin SVM? Describe the solution for very small and very large C. The perceptron criterion and the hinge loss are both zero for some correct examples: what does the "1" in $\max(0, 1 - z)$ change?

A15. With $m = 10$ classes, how many binary classifiers do one-versus-rest and one-versus-one train, and on how many examples each? What is an ambiguous region? How does each scheme resolve it, and can OvO voting still end in a tie?

A16. Explain why no linear classifier can solve XOR, and why training longer, or with a different linear algorithm, does not help. Name the three ways of obtaining non-linear features.

A17. Why does Algorithm 5 use the bias trick instead of updating an unpenalised bias b with the same step $\eta_k = 1/(\lambda k)$? How large is that step in the first iterations when $\lambda = 1/(nC)$?

B. Short calculations

B1. For $\mathbf{w} = (3, -4)$ and $b = 5$: classify $\mathbf{x} = (2, 1)$, compute its distance to the decision boundary, and the distance of the origin to the boundary. Give another $(\mathbf{w}, b)$ that defines the same classifier.

B2. Class -1 has points $(0,0)$, $(2,0)$, $(1,3)$ and class $+1$ has $(4,4)$, $(6,2)$, $(5,6)$. Compute the centroids, the nearest-centroid $\mathbf{w}$ and b, and classify $(4,2)$. Verify your answer by computing both distances.

B3. Run the perceptron by hand, from $\mathbf{w} = (0,0)$, $b = 0$, in the cyclic order $1,2,3,4$, on $\mathbf{x}_1 = (1,1)$, $t_1 = +1$; $\mathbf{x}_2 = (2,-1)$, $t_2 = -1$; $\mathbf{x}_3 = (2,2)$, $t_3 = +1$; $\mathbf{x}_4 = (3,0)$, $t_4 = -1$. Give a table as in Table 1 until a full epoch has no mistake, check that each update raises the margin of its example by $\|\mathbf{x}_i\|^2 + 1$, and give the final α_i of the dual form.

B4. Show that $\sigma'(z) = \sigma(z)(1-\sigma(z))$ and evaluate it at $z = 0$ and $z = 2$. For the three points $x = 0, 1, 2$ with labels $t = 0, 1, 1$ and $\tilde{\mathbf{w}} = (b, w) = (0,0)$, compute the gradient (4) and the parameters after one gradient step with $\eta = 0.5$.

B5. A three-class softmax gives scores $\mathbf{a} = (2, 0, -1)$. Compute the probabilities, the cross-entropy if the true class is the first, and the gradient of the loss with respect to the three scores. What happens to the probabilities if 5 is added to every score?

B6. For the SVM of the worked example in Section 5.2, $g(\mathbf{x}) = \frac{1}{3}(2x_1 + 4x_2 - 7)$, add the negative point $\mathbf{x}_5 = (2,1)$. With respect to this g, compute the functional and geometric margins of $\mathbf{x}_5$ and the slack $\xi_5 = \max(0, 1 - t_5\, g(\mathbf{x}_5))$ it would need. Does the old $(\mathbf{w}, b)$ remain a valid hard-margin solution once $\mathbf{x}_5$ is added? Using the definition of support vectors, explain why $\mathbf{x}_5$ must be a support vector of the new soft-margin solution when $C \geq 2$ (for such C the soft-margin solution of the four original points is the hard-margin one, since all its $\alpha_i \leq 10/9$).

B7. For the XOR points of Section 6 (class $+1$ at $(0,0)$ and $(1,1)$, class -1 at $(0,1)$ and $(1,0)$), add the feature $\phi_3 = (x_1 - x_2)^2$. Tabulate ϕ_3 on the four points and find (w_1, w_2, w_3, b) such that $g = w_1 x_1 + w_2 x_2 + w_3 \phi_3 + b$ is positive on class $+1$ and negative on class -1. Then explain why adding $\phi_3 = x_1 + x_2$ or $\phi_3 = x_1^2$ instead does not help.

B8. Continue the worked example "two Pegasos steps" (Section 5) with steps $k = 3$ and $k = 4$ of Algorithm 5, $\lambda = 1$, visiting $\mathbf{x}_3 = (3,1)$, $t_3 = +1$, and then $\mathbf{x}_2 = (0,0)$, $t_2 = -1$. Which of the four points are on the correct side of the resulting boundary?

C. Derivations

C1. Show that $\mathbf{w}$ is normal to the hyperplane $\mathbf{w}^\mathsf{T}\mathbf{x} + b = 0$ and derive the distance formula (1). Deduce the distance of the origin to the hyperplane and the width $2/\|\mathbf{w}\|$ of the band between $g(\mathbf{x}) = -1$ and $g(\mathbf{x}) = +1$.

C2. Derive ∇J for logistic regression using $\sigma' = \sigma(1-\sigma)$, and the Hessian $\sum_i y_i(1-y_i)\,\tilde{\mathbf{x}}_i\tilde{\mathbf{x}}_i^\mathsf{T}$. Conclude that J is convex, show that the Newton step is a weighted least-squares problem, and prove that $\|\tilde{\mathbf{w}}\|$ diverges on separable data.

C3. Prove the perceptron convergence theorem. Then show that the bound $(R/\gamma)^2$ does not change when all inputs are multiplied by the same constant, if the bias is absent.

C4. Derive the dual (6) of the hard-margin SVM from its Lagrangian. For the soft-margin problem, show that the dual is the same with the extra constraint $\alpha_i \leq C$.

D. Programming

D1. Implement the primal, averaged and dual perceptrons in numpy. On separable data of chosen γ and R compare the number of mistakes with $(R/\gamma)^2$; on non-separable data compare the last

iterate with the average.

D2. Implement Algorithm 4 in both variants. With $n = 10^4$, time both to a given loss as d grows from 10 to 1000, and explain the cross-over.

D3. Apply Algorithm 5 and least squares to two Gaussian classes plus a growing far-away cluster of correct points. Plot boundaries and errors; count the points with $t_i\, g(\mathbf{x}_i) \le 1$ for $C = 0.01, 1, 100$.

Further reading. Bishop [3], chapter 4 and section 7.1; Hastie, Tibshirani and Friedman [11], sections 4.3–4.5 and 12.2–12.3; for the history of the perceptron, Rosenblatt [15].

References

[1] G. Alain and Y. Bengio, Understanding intermediate layers using linear classifier probes, *ICLR Workshop* (2017).

[2] P. Baldi, P. Sadowski and D. Whiteson, Searching for exotic particles in high-energy physics with deep learning, *Nat. Commun.* 5, 4308 (2014).

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[4] B. E. Boser, I. M. Guyon and V. N. Vapnik, A training algorithm for optimal margin classifiers, *Proc. 5th Annual Workshop on Computational Learning Theory (COLT)*, 144–152 (1992).

[5] C. Cortes and V. Vapnik, Support-vector networks, *Mach. Learn.* 20, 273–297 (1995).

[6] D. R. Cox, The regression analysis of binary sequences, *J. R. Stat. Soc. B* 20, 215–242 (1958).

[7] K. Crammer and Y. Singer, On the algorithmic implementation of multiclass kernel-based vector machines, *J. Mach. Learn. Res.* 2, 265–292 (2001).

[8] R. A. Fisher, The use of multiple measurements in taxonomic problems, *Ann. Eugen.* 7, 179–188 (1936).

[9] Y. Freund and R. E. Schapire, Large margin classification using the perceptron algorithm, *Mach. Learn.* 37, 277–296 (1999).

[10] S. I. Gallant, Perceptron-based learning algorithms, *IEEE Trans. Neural Netw.* 1, 179–191 (1990).

[11] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[12] M. Minsky and S. Papert, *Perceptrons*, MIT Press (1969).

[13] A. B. J. Novikoff, On convergence proofs on perceptrons, *Proc. Symposium on the Mathematical Theory of Automata* 12, 615–622 (1962).

[14] A. Radford *et al.*, Learning transferable visual models from natural language supervision, *ICML* (2021).

[15] F. Rosenblatt, The perceptron: a probabilistic model for information storage and organization in the brain, *Psychol. Rev.* 65, 386–408 (1958).

[16] S. Shalev-Shwartz, Y. Singer and N. Srebro, Pegasos: primal estimated sub-gradient solver for SVM, *ICML* (2007).

[17] V. Vapnik and A. Lerner, Pattern recognition using generalized portrait method, *Automat. Remote Control* 24, 774–780 (1963).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.